

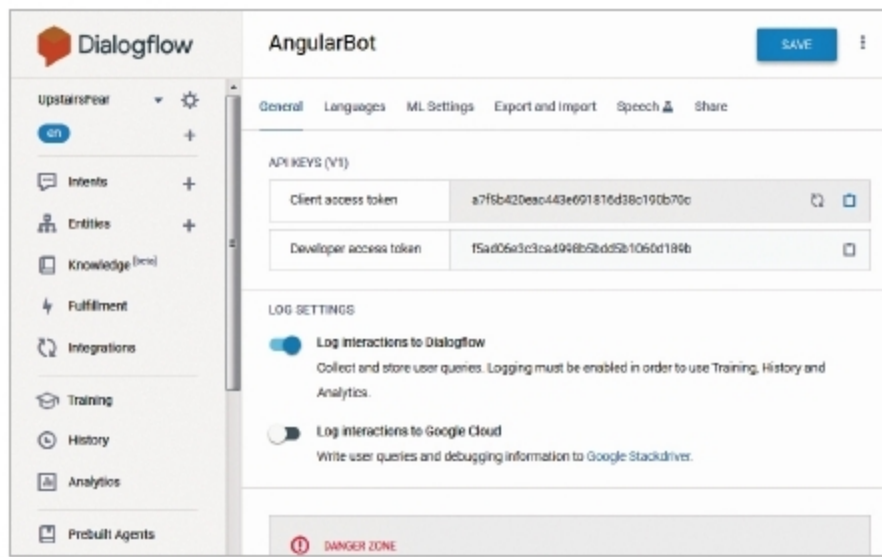


Figure 3: The client access token triggers intents by using their event field

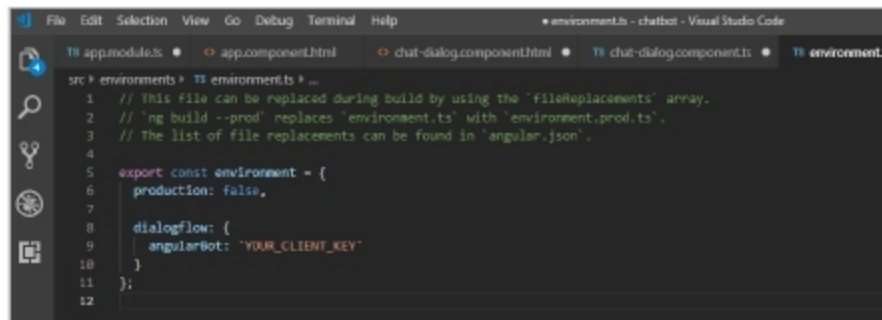


Figure 4: Pasting the client access key to the *environment.ts* file

Communicating with your bot (*chat.service.ts*)

To communicate with your bot, go into the Angular code and open *chat.service.ts*. You need to import the environment that you just added the API token to, as well as the API.AI client which was installed at the beginning. You need to set a read-only variable for the token and use that to initialise the API.AI client. You can set that as another variable called *client* and pass it to the API token.

To send a basic text request to the bot, you can set up a method called *talk()*, and call *client.textRequest()* with the text that you want to send within the parenthesis. The method sends a HTTP request to Dialogflow, which then responds with a JSON of the actual response from the agent.

chat.service.ts should look like what's shown below:

```
import { Injectable } from '@angular/core';
import { environment } from '../environments/environment';
import { ApiAiClient } from 'api-ai-javascript';
@Injectable()
export class ChatService {

  readonly token = environment.dialogflow.angularBot;
  readonly client = new ApiAiClient({ accessToken: this.token });

  constructor() {}

  talk() {
    this.client.textRequest(msg)
      .then( );
  }
}
```

You can set this chatbot up by going into your *chat-dialog.component.ts* file, injecting the chat service in the constructor and calling that *talk()* method which you just defined by adding the following command to the file:

```
this.chat.talk();
```

Making the bot user friendly (*chat.service.ts*)

To refactor things in order to make the bot a little more user friendly, you need to use *RxJS* to create an observable array of messages, and import *Observable* and *BehaviorSubject* from the library to the *chat.service.ts* file.

```
import { Observable } from 'rxjs/Observable';
import { BehaviorSubject } from 'rxjs/BehaviorSubject';
```

You also need to create a message class to define the way a message should look. The message class will have content, which includes a string and a *sentBy* property to distinguish between bots and humans.

```
constructor(public content: string, public sentBy: string) {}
```

Inside the chat service, you need to define a variable called *conversation*, which is a behaviour subject typed to an array of messages with a default value as an empty array.

```
conversation = new BehaviorSubject<Message[]>([]);
```

To make updates to this array, you need to create a function called *update*, which calls next on the behaviour subject. The function will clean up the code a little bit. Anytime you want to add a new message to the message feed, just call this method with the corresponding message object.

```
update(msg: Message) {
  this.conversation.next([msg]);
}
```

The conversation is going to start with a message from the user. That message is going to be in the form of a string. You need to convert the string into a message object. Thus, set the content equal to the message and *sentBy* property to the user.

```
converse(msg: string) {
  const userMessage = new Message(msg, 'user');
  this.update(userMessage);
}
```

To add this to the UI, you should call this update with the user message, and send the request to Dialogflow with that text, as follows:

```
return this.client.textRequest(msg)
```

As before, you can call the client text request to return a